Photo Booth — Project Documentation

A Java Swing re-creation of the 2005 Apple Photo Booth, with support for both local webcams and a phone-as-camera over Wi-Fi.

Table of Contents

1. Overview
2. Features
3. Requirements
4. Installation
5. Running the Application
6. User Guide
7. Phone Mode — Setup and Protocol
8. Effects Reference
9. Project Structure
10. Architecture
11. Class Reference
12. Build System
13. Batch Files
14. Git and Version Control
15. Networking and Firewall
16. Troubleshooting
17. Known Limitations
18. Extending the Project
19. Credits and License

1. OVERVIEW

Photo Booth is a desktop application written in Java that replicates the visual design and interaction model of the original Apple Photo Booth application, which shipped with the iMac G5 in October 2005. The original used the iMac's built-in iSight camera and its bright display as a fill-flash, and offered real-time visual effects ranging from simple color tints to complex facial distortions.

This implementation focuses on:

- **Visual fidelity to the 2005 original** — the 3×3 grid of live effect previews, the dot pagination indicator, the "Effects" button, the Polaroid slide-down animation, and the filmstrip carousel
- **Two frame sources** — a local USB webcam via the webcam-capture library, or any device that can POST JPEG frames over HTTP (typically a phone on the same Wi-Fi network)
- **Real-time pixel processing** — 18 effects implemented directly in Java, running at interactive frame rates

The application does not include the face-detection-based effects (Chipmunk, Frog, Dent, Dizzy, Space Alien, Bug Out) that the original had, because those require face-landmark detection and mesh warping beyond the scope of a pure-pixel implementation.

2. FEATURES

Live Video

- 3×3 grid of simultaneous live previews, one per effect on the current page
- 18 effects across 2 pages, switchable via dot pagination or the Effects button
- Mirrored feed — movements feel natural, like looking in a mirror
- 30 FPS rendering loop independent of source frame rate

Photo Capture

- 3-2-1 countdown overlay before the shutter fires
- 150 ms white flash to simulate fill-flash
- Optional shutter sound (drop a shutter.wav into resources)
- Polaroid slide-down animation — the captured photo ejects from the center of the grid and slides down into the filmstrip
- PNG saved to ./captures/ with a timestamp filename

Filmstrip

- Horizontal carousel of captured photos at the bottom of the window
- White paper border around each thumbnail, evoking a physical print
- Click any thumbnail to view the full-size image in a dialog
- Auto-scrolls to the newest photo after each capture

3. REQUIREMENTS

Runtime

Requirement	Version
Java	17 or newer (tested on Java 25, Eclipse Adoptium)
Operating System	Windows 10/11, macOS 10.15+, Linux with V4L2

Build

Requirement	Version
Maven	3.9 or newer

Optional

- A webcam accessible via webcam-capture
- A phone app capable of POSTing JPEG frames to an HTTP endpoint
- A shutter.wav file placed in src/main/resources/ for the shutter sound

4. INSTALLATION

Step 1 — Install Java
Download and install JDK 17 or newer from Eclipse Adoptium or any other JDK vendor. After installation, verify:

```
java -version
javac -version
```

Both should print a version 17 or higher.

Step 2 — Install Maven
Download the Binary zip archive of Maven from maven.apache.org/download.cgi. Extract it to a path without spaces — for example `C:\apache-maven\apache-maven-3.10.0`. Add the bin subfolder to your system PATH. Verify:

```
mvn -version
```

You should see the Maven version, Maven home, and your active Java version.

Step 3 — Clone or download the project

```
git clone https://github.com/milan-magar/photobooth.git
cd photobooth
```

Or download the source archive and extract it manually.

Step 4 — Build

```
mvn clean package
```

On first run, Maven downloads webcam-capture, bridj, slf4j, and their transitive dependencies. This takes one to three minutes depending on connection speed. When it finishes, you'll see `[INFO] BUILD SUCCESS` and the file `target/photobooth-1.0.0.jar` will exist.

5. RUNNING THE APPLICATION

Standard

```
java -jar target/photobooth-1.0.0.jar
```

With explicit native-access flags
If you're on Java 24 or newer and see warnings about restricted methods:

```
java --enable-native-access=ALL-UNNAMED --add-opens java.base/java.lang=ALL-UNNAMED -jar targ
```

6. USER GUIDE

Main window layout



Selecting an effect: Click any cell in the 3×3 grid. The selected cell gets a white border and a red name strip. The selected effect is the one used when you snap a photo.

Paging through effects: There are 2 pages of 9 effects each. The dots in the center of the bottom bar indicate the current page (blue = active, gray = inactive). Click a dot to jump directly to that page. Click the Effects button on the right to advance to the next page.

Taking a photo: Ensure the effect you want is selected. Click the round red shutter button in the center of the bottom bar. A large countdown appears over the grid: 3 → 2 → 1. The grid flashes white for 150 ms. A shutter click plays (if shutter.wav is present). A Polaroid-style print ejects from the center of the grid and slides down. The print lands in the filmstrip at the bottom. A PNG file is written to ./captures/.

Viewing a captured photo: Click any thumbnail in the filmstrip. A dialog opens with the full-size image. Click outside or press Escape to close.

Phone mode status: When running in Phone mode, the window title updates twice per second: `Photo Booth — Phone (port 8080) — 47 ms ago | received=127`. If it says `waiting for frames...`, no phone is currently sending.

7. PHONE MODE — SETUP AND PROTOCOL

How it works: The Java app opens an HTTP server on port 8080, bound to all network interfaces (0.0.0.0:8080). Your phone connects to the same Wi-Fi as the laptop. Your phone sends JPEG frames to the laptop via POST /upload. Each received frame is decoded, mirrored, and pushed to the 3×3 grid.

Server endpoints

Method	Path	Purpose
POST	/upload	Accept a JPEG frame (raw body, Content-Type: image/jpeg recommended but not required)
GET	/latest	Returns the newest frame as image/jpeg, or 204 No Content if none yet
GET	/	A small HTML landing page showing the frame counter and a manual upload form

Phone-side examples

curl / shell:

```
curl --data-binary @photo.jpg http://192.168.1.42:8080/upload
```

Loop at ~15 fps:

```
while true; do
  curl -s --data-binary @snap.jpg http://192.168.1.42:8080/upload > /dev/null
  sleep 0.066
done
```

Android (Kotlin):

```
val url = URL("http://192.168.1.42:8080/upload")
val conn = url.openConnection() as HttpURLConnection
conn.requestMethod = "POST"
conn.doOutput = true
conn.setRequestProperty("Content-Type", "image/jpeg")
conn.outputStream.use { it.write(jpegBytes) }
conn.responseCode  // 200 on success
```

8. EFFECTS REFERENCE

Page 1

Effect	Description
Sepia	Warm brown vintage tone using the standard luminance-weighted matrix
Black and White	Grayscale using perceived luminance (0.299R + 0.587G + 0.114B)
X-Ray	Inverted colors with a contrast stretch and a cool blue tint
Thermal Camera	Luminance mapped to a Deep Blue → Purple → Red → Yellow → White gradient
Pop Art	2×2 quadrant with four distinct color shifts (Warhol style)
Glow	Highlights boosted to create a bloom effect
Rose Tint	Warm red/pink push with reduced green
Plastic Camera	Oversaturated, warm, high-contrast — resembling a cheap toy camera
Normal	Unmodified mirrored feed

Page 2

Effect	Description
Bulge	Outward radial distortion: r_new = r × √(r / r_max)
Squeeze	Inward radial distortion: samples pulled toward center
Twirl	Rotational displacement; angle increases toward the center
Fish Eye	Wider-angle curvature than Bulge, using an atan(r) mapping
Stretch	Horizontal compression with vertical expansion
Mirror	A second horizontal flip (net effect: unmirrored)
Light Tunnel	Concentric radial rings from power-curve sampling
Halftone	4×4 dot pattern, black ink on white paper
Invert	Full color inversion (photographic negative)

9. PROJECT STRUCTURE

```
photobooth/
├── pom.xml
├── README.md
├── .gitignore
├── build.bat
├── run.bat
├── start.bat                    (optional)
├── LICENSE                      (optional)
├── src/
│   └── main/
│       ├── java/
│       │   └── com/
│       │       └── photobooth/
│       │           ├── PhotoBoothMain.java
│       │           ├── FrameSource.java
│       │           ├── WebcamFrameSource.java
│       │           ├── HttpFrameSource.java
│       │           ├── CaptureWorker.java
│       │           ├── FilterEngine.java
│       │           ├── EffectsGridPanel.java
│       │           ├── ShutterBar.java
│       │           ├── ShutterButton.java
│       │           └── FilmStripPanel.java
│       └── resources/
│           └── shutter.wav      (optional)
│
```

11. CLASS REFERENCE

PhotoBoothMain: Entry point. Presents the source-selection dialog, creates the JFrame, wires up panel tree, starts streaming. Key methods: `main(String[])`, `run()`, `chooseFrameSource()`, `createWebcamSource()`, `createPhoneSource()`, `findLanIp()`.

FrameSource: Interface implemented by all frame providers.

```
void start();
void stop();
BufferedImage getLatestFrame();
String getDescription();
```

`getLatestFrame()` must be non-blocking. Returns null if no frame is available yet.

WebcamFrameSource: Wraps a `com.github.sarxos.webcam.Webcam`. Opens device on start(), closes on stop(), converts frames to TYPE_INT_RGB.

HttpFrameSource: Binds to 0.0.0.0:8080 and exposes endpoints. Stores latest decoded frame and raw JPEG bytes. Provides status counters. Constants: PORT=8080, UPLOAD_ENDPOINT="/upload", LATEST_ENDPOINT="/latest", MAX_FRAME_BYTES=4MB.

CaptureWorker: A `SwingWorker<Void, BufferedImage>` that polls FrameSource at 30 fps and publishes frames to EDT. Stops gracefully on cancel().

FilterEngine: Pure static utility class. Holds Effect enum and methods per transformation: `apply(BufferedImage, Effect)`. Color effects: sepia, xray, thermal, popArt, glow, halftone, roseTint, plasticCamera, blackAndWhite, invert. Geometric: bulge, squeeze, twirl, lightTunnel, fisheye, stretch, mirrorHoriz.

EffectsGridPanel: Central 3×3 grid. Manages current page, selection, countdown overlay, white flash, and Polaroid slide-down animation. Holds PAGE_1 and PAGE_2 static arrays. Key methods: `startStreaming()`, `updateFrame()`, `captureSelected()`, `setPage()`, `showCountdown()`, `showFlash()`, `startPolaroidAnimation()`.

ShutterBar: Middle control bar with caption, dot pagination, shutter button, Effects button. Owns capture sequence via `triggerCapture()`, `registerPolaroidCallback()`, `rebuildDots()`.

ShutterButton: Custom JButton painting a round metallic ring with a red center and specular highlight.

FilmStripPanel: Horizontal carousel of captured photos with white paper borders. Clicking thumbnail opens full-size modal. Method: `addThumbnail(BufferedImage)`.

12. BUILD SYSTEM

pom.xml highlights: Dependencies (webcam-capture 0.3.12, slf4j-api, slf4j-simple), maven-compiler-plugin (Java 17), maven-shade-plugin (bundles runnable shaded JAR with mainClass com.photobooth.PhotoBoothMain and native-access manifest entries).

Build output: `target/photobooth-1.0.0.jar` (runnable shaded JAR ~5MB), `target/classes/`, `target/maven-status/`.

Build commands: `mvn clean` (delete target/), `mvn compile`, `mvn package`, `mvn clean package` (recommended), `mvn dependency:tree`.

13. BATCH FILES

build.bat: Runs `mvn clean package`, checks Maven on PATH and pom.xml. Pauses on error.

run.bat: Checks JAR existence and launches with explicit native-access flags:

```
java --enable-native-access=ALL-UNNAMED --add-opens java.base/java.lang=ALL-UNNAMED -jar targ
et\photobooth-1.0.0.jar
```

start.bat: Optional script that builds first if JAR is missing, then runs.

14. GIT AND VERSION CONTROL

What to commit: .gitignore, README.md, LICENSE, pom.xml, build.bat, run.bat, start.bat, src/main/java/com/photobooth/*.java, src/main/resources/shutter.wav.

What never to commit: target/, captures/, .idea/, *.iml, .vscode/, .classpath, .project, .settings/, *.class, *.log, .DS_Store, Thumbs.db, dependency-reduced-pom.xml.

Standard workflow & Untracking:

```
git add . && git commit -m "describe changes" && git push
# If target/ or captures/ were accidentally tracked:
git rm -r --cached target captures
git add .gitignore && git commit -m "Untrack build output" && git push
git ls-files | findstr /i "target captures"  # Verify clean output
```

15. NETWORKING AND FIREWALL

Port binding: Server binds to `0.0.0.0:8080` (all interfaces). Verify via `netstat -an | findstr :8080` (should list 0.0.0.0:8080 LISTENING).

Windows Firewall commands:

```
netsh advfirewall firewall add rule name="PhotoBooth 8080" dir=in action=allow protocol=TCP l
ocalport=8080
netsh advfirewall firewall delete rule name="PhotoBooth 8080"
```

16. TROUBLESHOOTING

- **"java is not recognized" / "mvn is not recognized":** Add JDK bin / Maven bin subfolders to system PATH.
- **NoClassDefFoundError (Aether):** Spaces in Maven install path or version mismatch. Move Maven to a space-free path (e.g. C:\apache-maven\).
- **Webcam not detected:** Grant camera permissions to Terminal/IDE in macOS Settings or Windows Privacy. Ensure no Zoom/Teams session is holding camera. On Linux verify /dev/video0 exists.
- **Phone mode "waiting for frames...":** Verify laptop IP reachable in phone browser (http://laptop-ip:8080/). Ensure raw JPEG bytes sent to /upload.
- **UnsatisfiedLinkError (Apple Silicon):** Run under Rosetta: `arch -x86_64 java -jar target/photobooth-1.0.0.jar`.
- **Port 8080 in use:** Find PID via `netstat -ano | findstr :8080` and kill via `taskkill /PID <pid> /F`.

17. KNOWN LIMITATIONS

Missing face-landmark effects (Chipmunk, Frog, Dent, Dizzy, Space Alien, Bug Out require OpenCV/MediaPipe); non-fullscreen flash; single-page scrolling filmstrip; no MP4/GIF video recording; shutter sound depends on optional shutter.wav; manual IP entry on phone.

18. EXTENDING THE PROJECT

Adding a new effect: Add entry to `Effect` enum in FilterEngine.java → Implement static pixel method using `int[] px` manipulation → Add effect to EffectsGridPanel.PAGE_1 or PAGE_2 → Rebuild.

Adding mDNS discovery: Add `org.jmdns:jmdns:3.5.9` to pom.xml and register service `_photobooth._tcp.local.` on server start.

19. CREDITS AND LICENSE

Inspired by the original 2005 Apple Photo Booth. Third-party libraries: `webcam-capture` (MIT), `bridj` (BSD), `slf4j` (MIT). Entire implementation released under the MIT License.